Graph

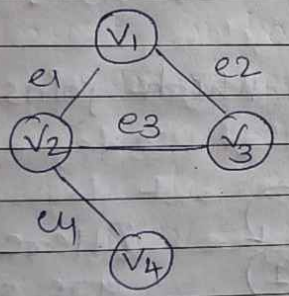
DATE
PAGE

2→ 2D - array

2→ Non-linear data structure
(I may or may not travel whole list)

2→ Connection of vertices and edges

2→ A graph consist of non-empty set 'V' called set of nodes (points or vertices) of the graph, a set 'E' which is the set of edges and a mapping from set of edges to a set of pairs of elements of 'V'.



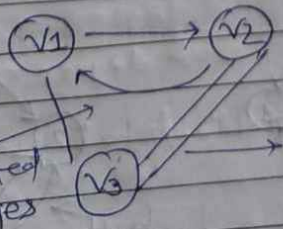
V = 4 vertices (V1 to V4)
E = {e1, e2, e3, e4}
e1 → (V1, V2)
e2 → (V1, V3)
e3 → (V2, V3)

- (V1) : This is a graph with no edge

(V1) (V2) : Graph with no edge

(V1) → (V2) : Directed graph / Digraph
(Have direction)

2→



known as "Mix Graph"

→ parallel edges
(not-direction)

→ $V_1 \longrightarrow V_2$ (V_1, V_2)

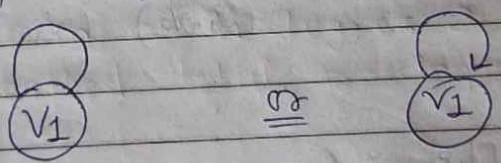
V_1 : Initial vertex
V_2 : terminal vertex

→ V_2 —— V_1 (No direction)

Bidirectional graph
(V_1, V_2) or (V_2, V_1)

→ An edge is initiating or originating in the node V_1 ($V_1 \longrightarrow V_2$) and terminating in node V_2, node V_1 is called initial vertex (node) and V_2 is called terminal vertex (node).

→ An edge of the node which joins itself is called loop or sling

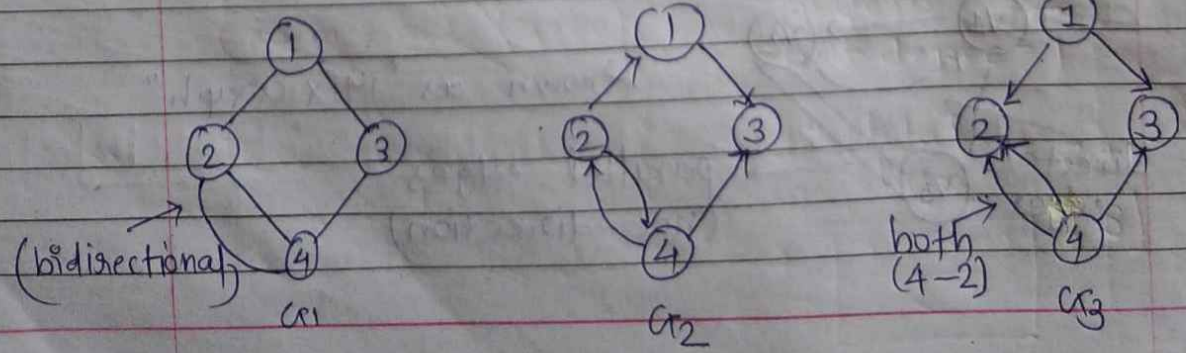


[Here both are same]

Undirected Directed

— No meaning of direction in self loop

→ If 2 Node are join by more than 1 edge such edges are called 'parallel edge'



Here e1 & e3 are parallel edge.

→ If a graph contains any parallel edge then it is called multi graph.

→ If a graph does not contain any parallel edge then it is called simple graph.
(G2 is simple graph)

→ Parallel edges are useful like short path, long path.

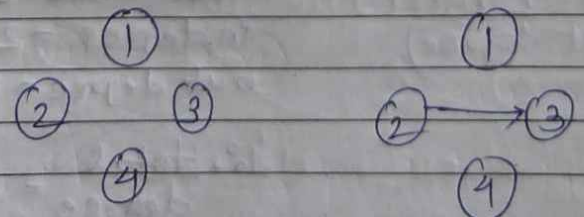
→ A graph in which weights are assigned to every edge is called weighted graph.

- it can be negative.

→ If loop ((V1)) → simple graph.

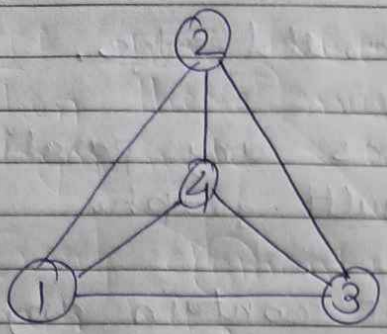
→ A node which is not adjacent to any other node is called isolated node/vertex.

→ A graph containing only isolated nodes is called null graph.

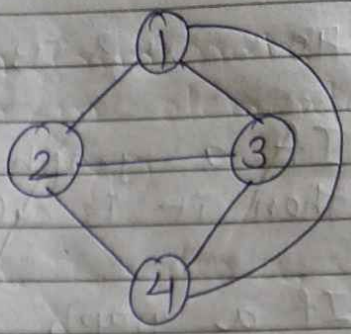


Not a null graph
(have edge).

2 →



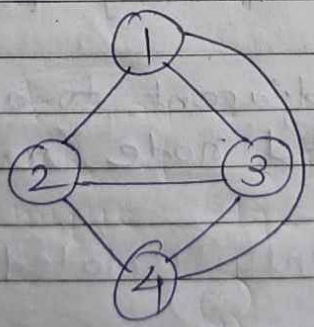
G2



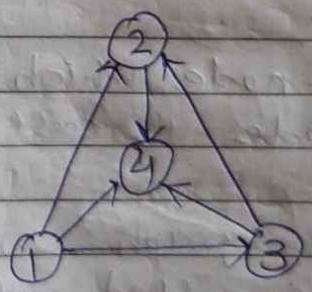
G1

G1 & G2 are same graph, but representation is different → isomer figure

2 → In a directed graph for node V:, no of edges which have 'V'
- Initial node is called out degree of V
- Terminal node is called In degree of V
- sum of in degree & out degree is called total ~~or~~ degree V.



Undirected graph
(no in & out degree)

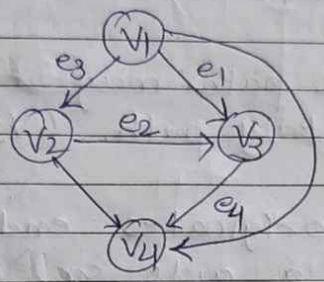


For node-2
In degree = 2
out degree = 1

i.e. total degree = 2+1
= 3

2→ Path : path is a sequence of edges of a graph such that the terminal node of any edge in the sequence is initial node of next edge, if any in the sequence.

eg



not path :
$((v_1, v_2), (v_3, v_4))$
↑ terminal

path :
$((v_1, v_2), (v_2, v_4))$
same

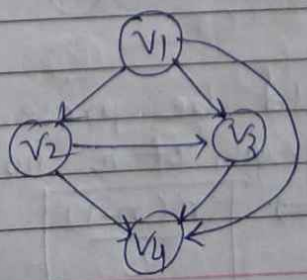
2→ No. of edges appearing in the path is called length of the path.

$((v_1, v_2), (v_2, v_4))$ ⟶ length = 2
① ②

(e_3, e_2, e_4) ⟶ length = 3

2→ A path in which edges are distinct is called simple path.

2→ A path in which all the nodes through which it traverses are distinct is called an elementary path.



DATE:
PAGE:

→ If there exist a path from node x to y, then there must exist an elementary path from node x to y.

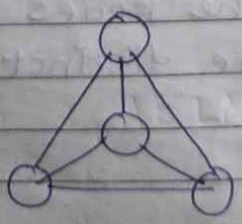
→ Every elementary path is also a simple path but,
Every ~~el~~ simple path may or may not always elementary path. (V_1, V_2, V_3, V_1)

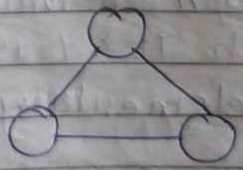
→ A path which originates ends in the same node is called a cycle or 'circuit'.

→ Acycle is called elementary acycle if it does not traverse through any node more than once (ignore the ends)

→ A graph which does not have any cycle is called 'acyclic'.

→ A simple undirected graph in which every pair of distinct vertices is connected by unique edge is called complete graph.

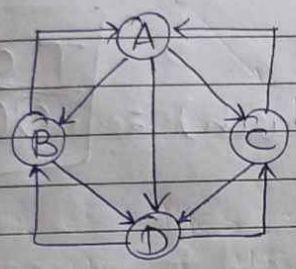




→ How many edges would be present in undirected complete graph of n nodes (ignore the loops) ? ⟹ $\boxed{\frac{n(n-1)}{2}}$

2 → A simple directed graph in which every pair of distinct vertices is connected by a pair of unique edges (one in each direction) is called complete graph.
- no. of edges in directed graph: $n(n-1)$.

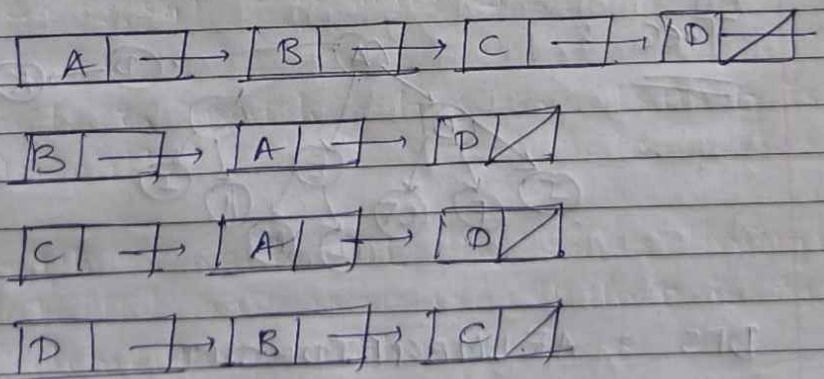
* Adjacency Matrix :



	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

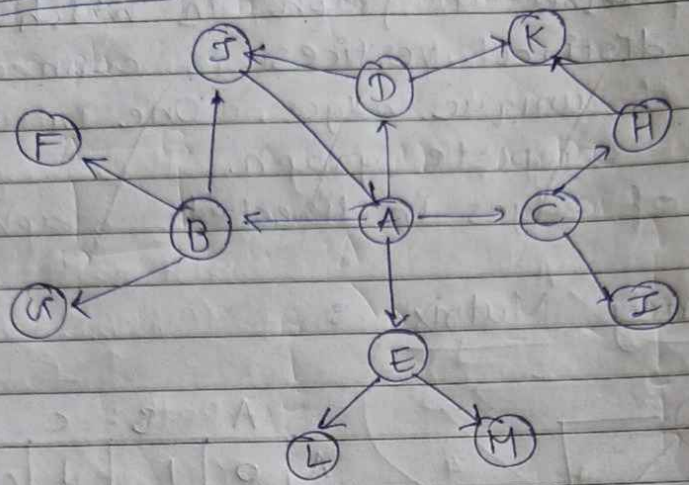
Row: out degree
column: indegree

OR Linked List



DATE:
PAGE:

Traversal:



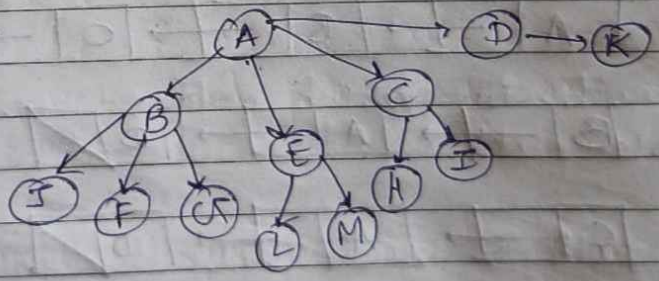
T.C. for both
$O(V+E)$

2 → BFS (Breadth first search) : ABECDJFGLMHIK

~~ABECDJFGLMHIK~~

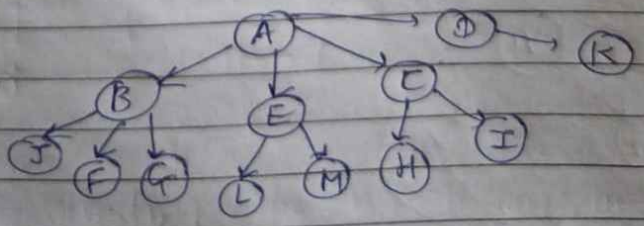
visited [ ] = A B C D E F G H I J K L M
1 1 1 1 1

Tree :



2 → DFS : AELMBGFJDKCHI

Tree :



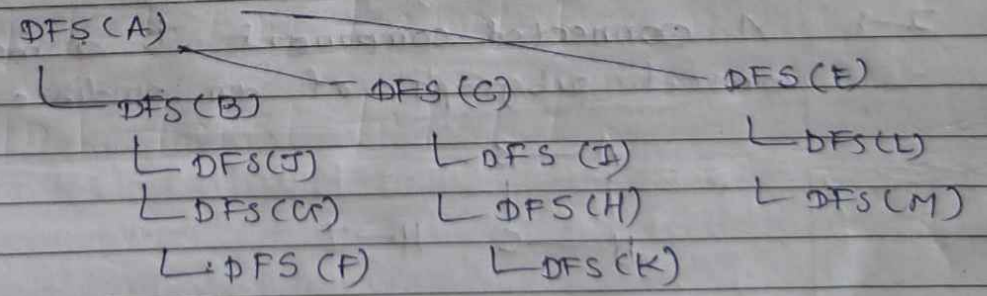
Algorithm :-

2 → BFS (G, S) // G is graph, S is source node.

```
{
  let Q be a queue
  Q.enqueue(S)
  mark S as visited
  while Q is not empty
      S = Q.dequeue() ; print(S.val)
    for all adjacent w of S in graph G
      if w is unvisited then
          Q.enqueue(w)
          mark w as visited
}
```

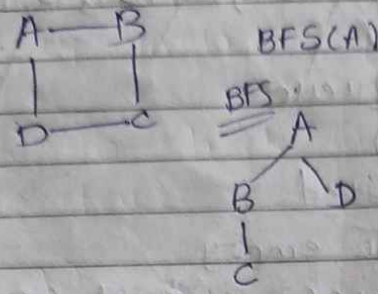
2 → DFS (V)

```
{
  visited[V] = 1
  for all adjacent vertices w of V
                  If w is not visited
     DFS(W)
}
```



2→ shortest path for undirected non-weighted graph.

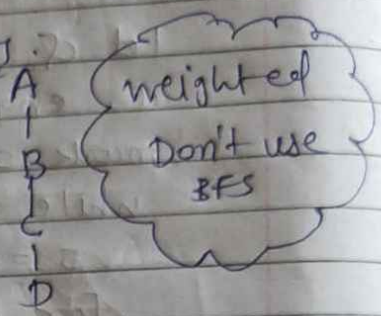
Use BFS



BFS(A) = ABDC

BFS:

DFS:

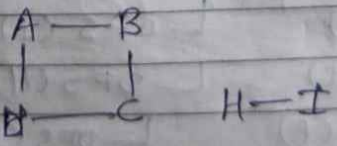


2→ To check whether there is cycle in given graph or not → BFS & DFS

2→ An undirected graph is said to be connected if there is a path bet^{n} every pair of vertices.

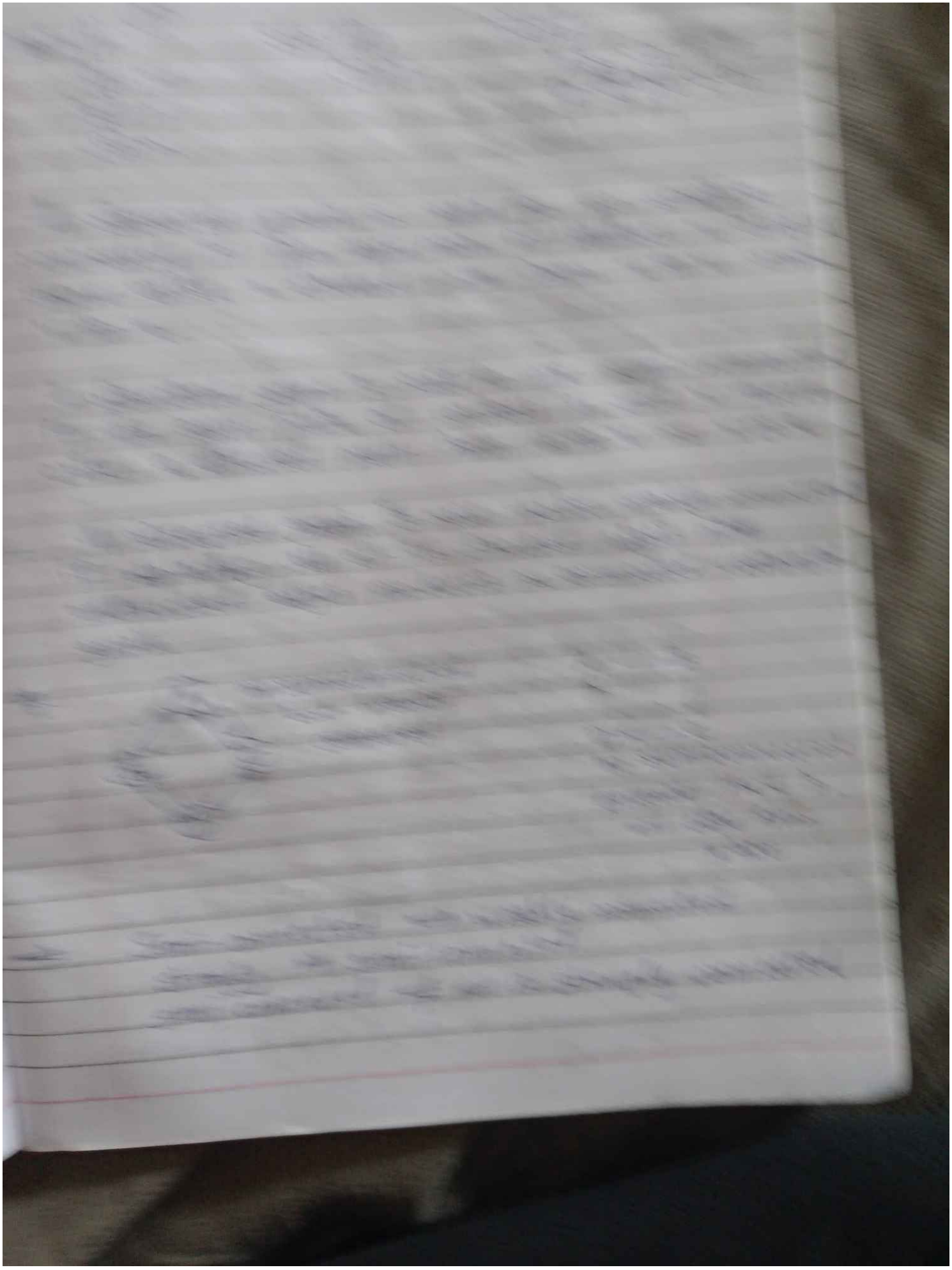
2→ An undirected graph is said to be disconnected if there exist two vertices which are not end points of any path in graph.

2→ A connected componant is a maximal connect subgraph of an undirected graph



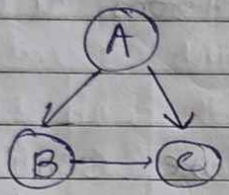
2→ T.C. of finding the how many ~~no of times~~ components are there in given graph

O(V(V+E))



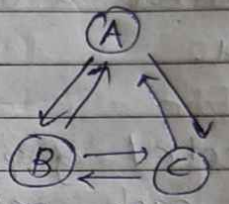
2 → If we convert all directed edges into undirected edges and apply BFS or DFS and if we get all the vertices then we can say given graph is weakly connected graph

Ex. How to check given graph is semi-connected?



ABC

semi connected



ABC

Strongly connected

⇓

- BFS / DFS through all vertices, till find vertex through which we can find all vertices

For strongly connected:
Apply BFS/DFS for each vertices, if all vertices traversal gives all vertices then it is strongly connected.

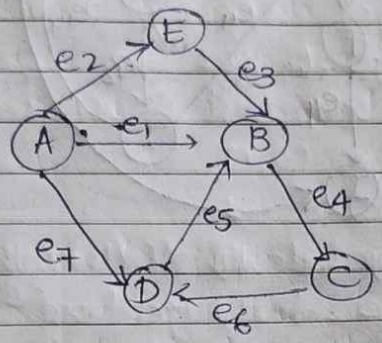
⟹ Deleting a cycle:

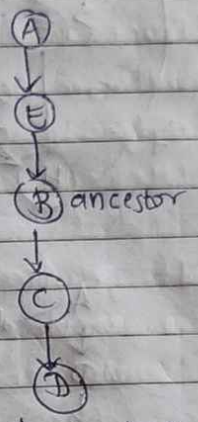
Step 1: All vertices are coloured white at the beginning.

step 2: colour of vertex will change from white to grey. (when it is visited 1st time)

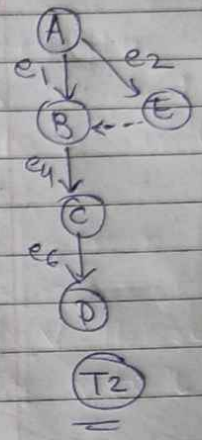
→ Cross edge : It is an edge which connects 2 nodes such that there do not have any ancestor and discendent relationship bet^{n} them

(1)





T1



AEBCD

Consider T2

not relation bet^{n} B & E

u	v
E	B

e_3 = cross edge

e_5 =

e_7 forward edge

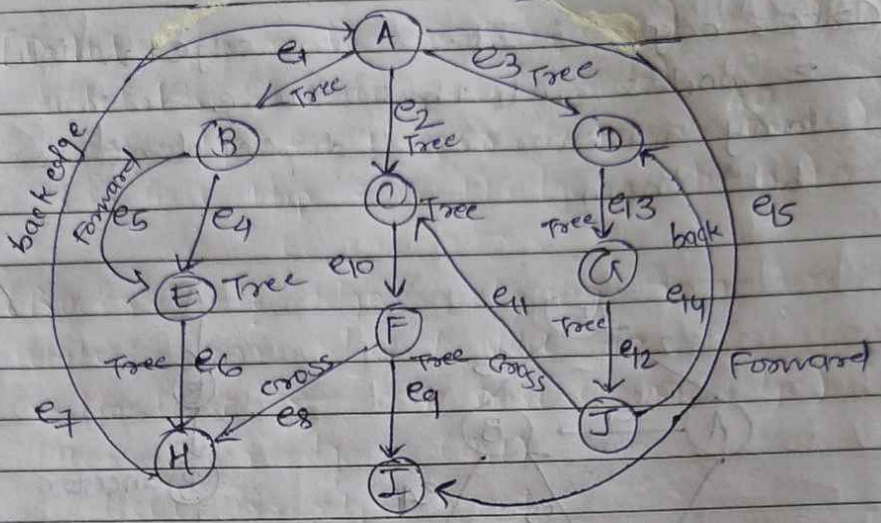
u	v
A	D

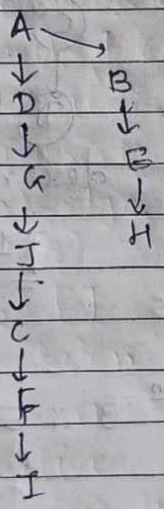
$A \xrightarrow{e_7} D$ Here v is descendent

D → B | ancestor
u | v | back edge

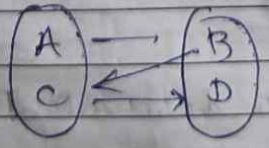
e5 back edge represent cycle.

(2)





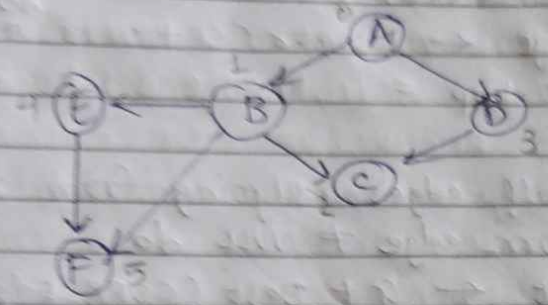
→ No edge between vertices of same group, then it is called Bivertice edge.



Bivertice edge
Bipartite

How to check given graph is bivertice or not.

- Representation of graph using adjacency list



vertex

data	visited	Listptr

Edge

Dest	Next

G

0	A	false	→ 1 → 3
1	B	false	→ 2 → 4 → 5
2	C	false	
3	D	false	→ 2
4	E	false	→ 5
5	F	false	

BFS Algo:

```
BFS (index)
index = starting node for BFS
G = pointer to adjacency list
curr_node = temp integer variable
curr_edge = pointer to edge node
Edge_dest = pointer to an element of array
```

(1) [Initialize the queue]
visited (G+index) ← true
write (DATA (G+index))
Insert (queue, index)

(2) [Process nodes from queue till its empty]
Repeat while queue is not empty

③ [Delete a node from queue]
Qdelete (Queue, curr_node)
curr_edge ← Listptr (G+curr_node)
(in first loop = (0,1) edge it returns)

④ [Process all edges outgoing from curr_node]
While curr_edge ≠ NULL do
Edge_dest ← G+Dest (CURR_Edge)
(Here curr_edge = (0,1)
dest (curr_edge) = 1
If visited (Edge_dest) = false then
visited (edge_dest) ← true
Write (DATA (edge_dest))
Qinsert (Queue, DEST (curr_edge))
curr_edge ← NEXT (curr_edge)

⑤ [Finished]
return.

DFS Iterative

Algo^r : DFS (index)
index = starting node for DFS
G = pointer to adjacency list

① [Initialize the stack]
PUSH (stack, index)

② [Process nodes from stack is not empty]
(repeat while stack is not empty)

③ [Pop a node from stack and process it]
POP (stack, CURR_node)
If visited (cr+ CURR_node) = true then
go to step 2
visited (cr+ CURR_node) ← true
Write (DATA (cr+CURR_node))
CURR_edge ← listptr (cr+CURR node)

④ [Add all edges outgoing from current node to stack]
While CURR_edge ≠ NULL do
Edge_dest ← cr+ DEST (CURR_edge)
if visited (edge_dest) = false then
PUSH (stack, Dest (CURR_edge))
CURR_edge ← NEXT CURR_edge)

⑤ [Finished]
return

⇒ Recursive
Algo^r : DFS_RECURSIVE (index)

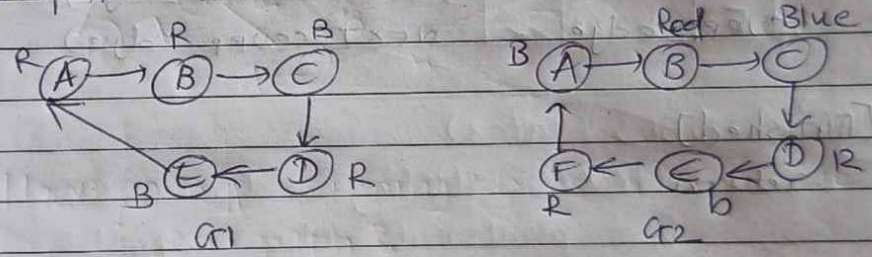
(1) [Process vertex represented by index]
If visited (cr+index) = true then
return
visited (cr+index) ← true
Write (DATA (cr+index))

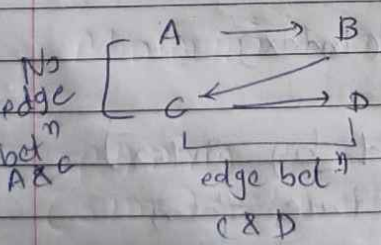
(2) [Recursively call for all adjacent nodes]

curr_edge ← listptr (&curr_node)
While curr_edge ≠ NULL do
  Edge_dest ← &Dest (curr_edge)
  if (visited (edge_dest) = false) then
    DFS_RECURSIVE (DEST (curr_edge))
  curr_edge ← NEXT (curr_edge)

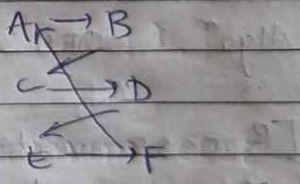
(3) [Finished]
return

⟹ If vertices can be divided into two sets such that there is no edge bet^n two vertices of same set, then it is called Bipartite graph.





No bipartite graph

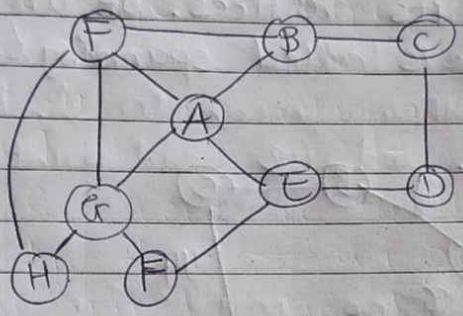


Bipartite graph

T.C. $O(V+E)$

2→ Biconnected Graph:

→ A vertex in undirected graph whose removal will split the graph into two or more componants is called 'cut vertex' / Articulation point.



A, C are cut vertexes.

→ Application:
- A Network in which no cut vertex is called Biconnected graph.

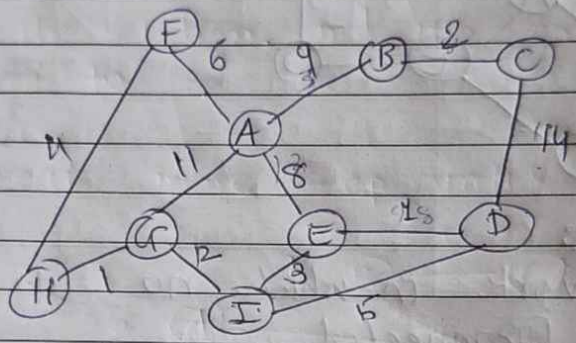
In
- Network, ideally there should be no cut vertex.

→ Remove one vertex → Apply DFS/BFS
if we get all the vertices then it is Biconnected
- we have to check for all vertices.
O(V(V+E))

DATE:
PAGE:

* Spanning Tree :

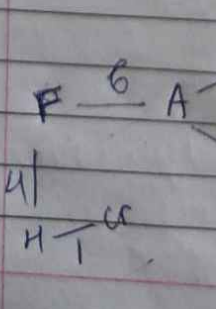
- Connected Acyclic graph is called Tree.
- connected subgraph without cycles which includes all its vertices of undirected graph G(V, E) is called spanning tree.



Ans. How many spanning trees are possible for given graph ?

- Minimum spanning tree (MST).
- spanning tree with minimum cost (sum of weights of edges included in spanning tree) is called MST.

Kruskal's Algo ~ MST



edges (sorting)

ED	1 ✓	AF	6 ✓
HG	1 ✓	AE	8 ✓
BC	2 ✓	AB	9 ✓
EI	3 ✓	AG	11 ✗
HF	4 ✓	GI	12 ✗
ID	5 ✗	ED	14 ✗

A	B	C	D	E	F	G	H	I
1	2	3	4	5	6	7	8	9
8	8	9	5	9	8	8	9	
9	9		9		9	9		

When all of them are highest we have to stop. (it means all of them are connected)

a) Replace with max.

If values are same we have to ignore that edge

$O(V^2 + E \log E)$

V^2 → array.

$E \log E$ → sorting

Steps:

(1) Egdes are sorted in ascending order of ~~that~~ their weights.

(2) Sets are used to detect cycle in the tree. Initially, each vertex has its own set.

(3) Picks edges one by one in ascending order.

(4) If two end points of edges are in the same set then ~~informs~~ it forms a cycle otherwise edge is added to the spanning tree and sets corresponding ~~to~~ end-points are combined.

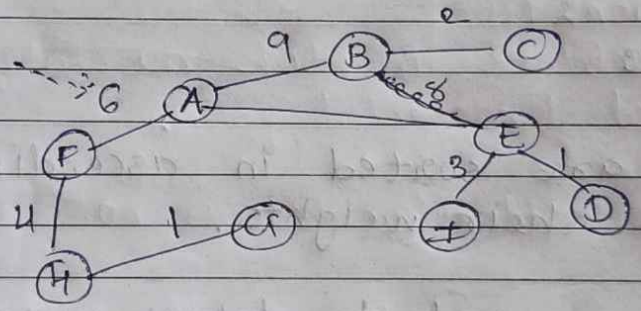
DATE:
PAGE:

Prim's Algo^{n} :

(1) Cut the graph into two sets S and S' a vertex is picked at random. is and is added to S. rest of the vertices are part of S'.

(2) Smallest edge that is connecting a vertex in S. and another vertex in S'. is added to the solution. and its end point in S' is moved to S.

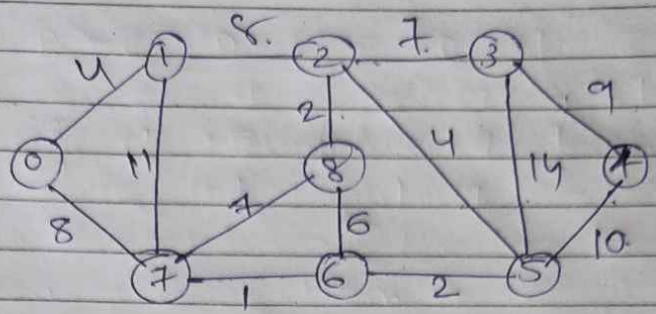
(3) Repeat above step until S' is empty.



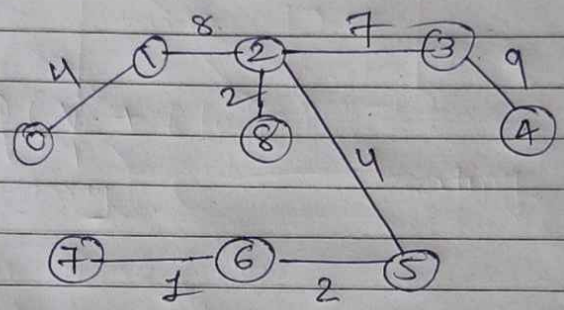
S' = {A, B, C, D, E, F, G, H, I}
S = {F} ← Initially

$O(V^2 + E \log E)$

Eg:



Kruskal:



cost : 37

76 → 1 ✓
65 → 2 ✓
28 → 2 ✓
01 → 4 ✓
25 → 4 ✓
86 → 8 X
78 → 7 X
23 → 7 ✓
12 → 8 ✓
07 → 8 X
34 → 9 ✓
45 → 10 X
17 → 11 X
35 → 14 X

Prim: